PRACTICAL APPLICATION:
4. Alert Triage with Automation:

Triage Simulation:
Analyzing mock alert in wazuh:

1) Create a Test File to Simulate a Suspicious Download:
    Create the file
        mkdir -p ~/Downloads
        touch ~/Downloads/suspicious-file.exe

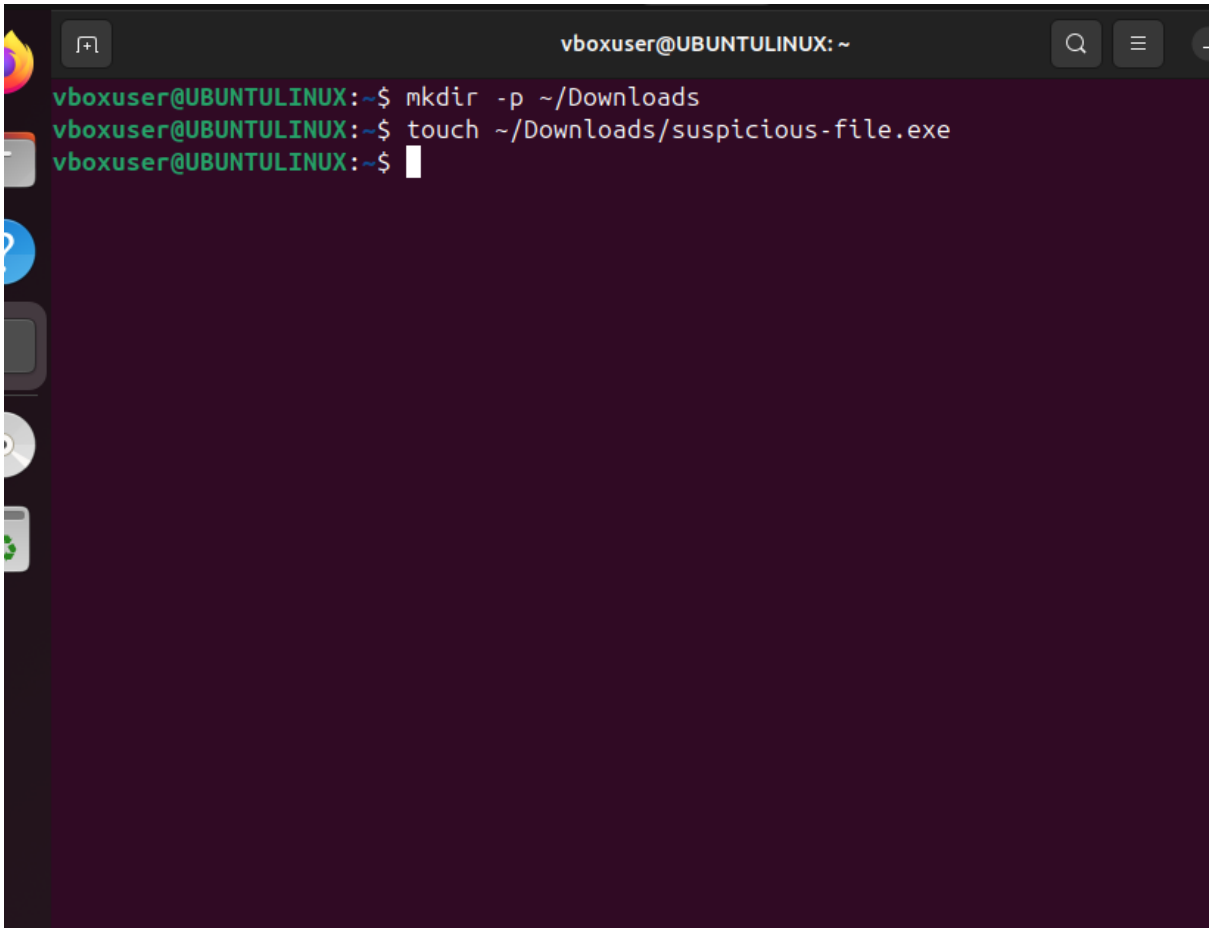


Figure 1: created test file

Generate test file SHA256 hash,
    sha256sum ~/Downloads/suspicious-file.exe

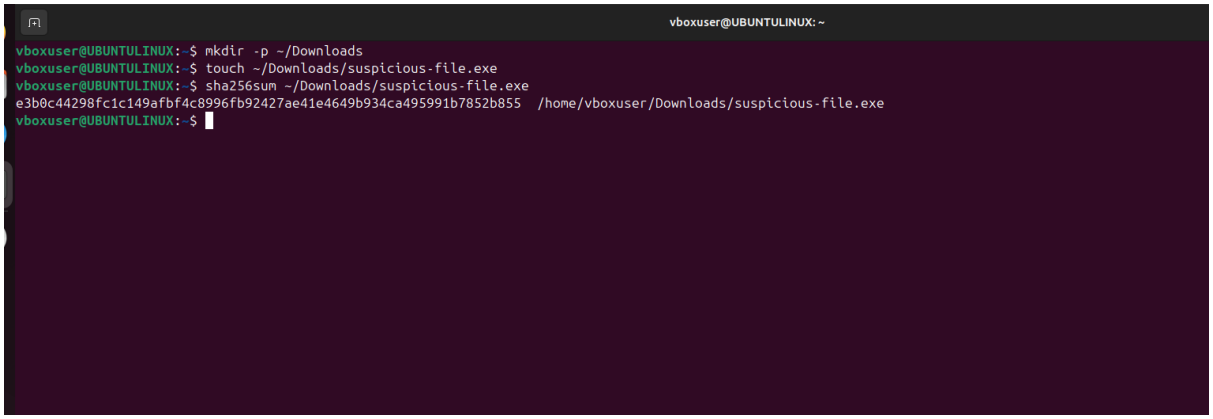


Figure 2: hash of the test file

2) Add a Custom Wazuh Rule to Detect the File:

Open local rules file ,

sudo nano /var/ossec/etc/rules/local_rules.xml

and add below shown custom rules ,

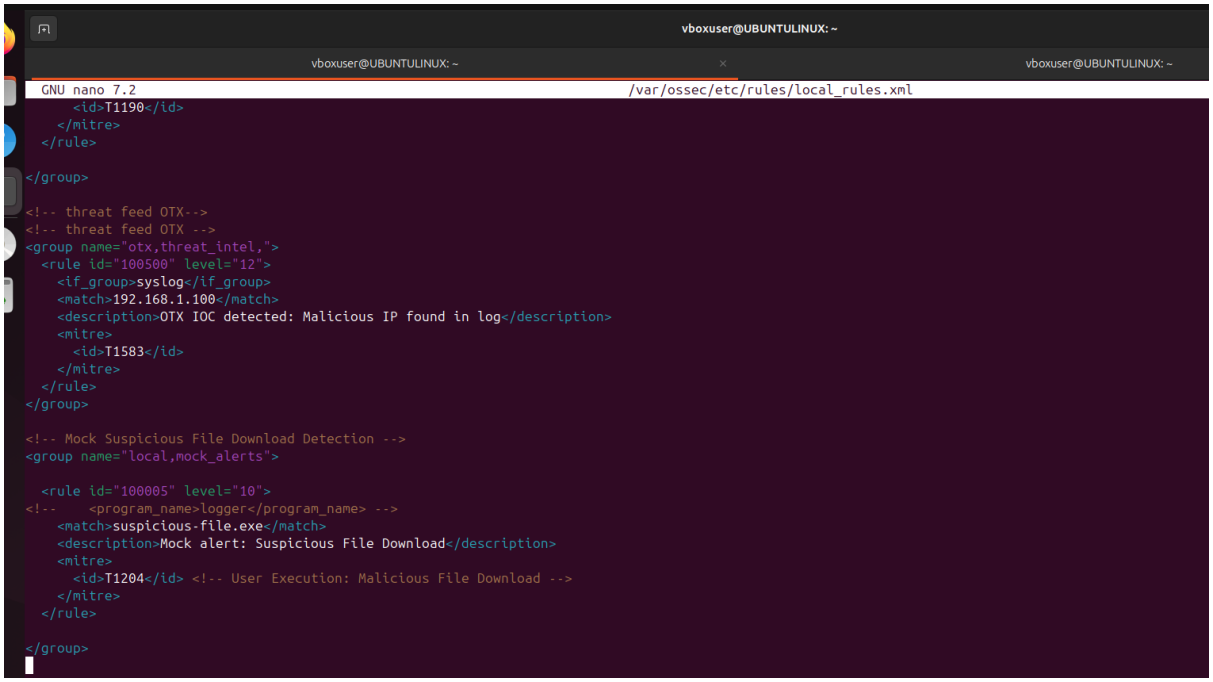


Figure 3: add custom rule on local rules file

Then restart the wazuh manager ,

sudo systemctl restart wazuh-manager

```
vboxuser@UBUNTULINUX:~$ sudo nano /var/ossec/etc/rules/local_rules.xml
[sudo] password for vboxuser:
vboxuser@UBUNTULINUX:~$ sudo nano /var/ossec/etc/rules/local_rules.xml
vboxuser@UBUNTULINUX:~$ sudo systemctl restart wazuh-manager
Warning: The unit file, source configuration file or drop-ins of wazuh-manager.service
changed on disk. Run 'systemctl daemon-reload' to reload units.
vboxuser@UBUNTULINUX:~$ sudo systemctl daemon-reload
vboxuser@UBUNTULINUX:~$ sudo systemctl restart wazuh-manager
vboxuser@UBUNTULINUX:~$
```

Figure 4: restarted wazuh manager

3) Trigger the Alert:

simulate the alert by writing a log that Wazuh can detect

logger "Downloaded suspicious-file.exe from 192.168.1.102"

logger "Downloaded suspicious-file.exe from 192.168.1.103"

```
vboxuser@UBUNTULINUX:~$ sudo systemctl daemon-reload
sudo systemctl restart wazuh-manager
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.103"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.103"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.103"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ sudo nano /var/ossec/etc/rules/local_rules.xml
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$ logger "Downloaded suspicious-file.exe from 192.168.1.102"
vboxuser@UBUNTULINUX:~$
```

Figure 5: triggering alert

4) Verify the Alert in Wazuh Dashboard

Access wazuh through browser using,

https://10.0.2.4:443

and goto threat hunting and in search bar type ,

rule.id:100005

then analyze the alert.

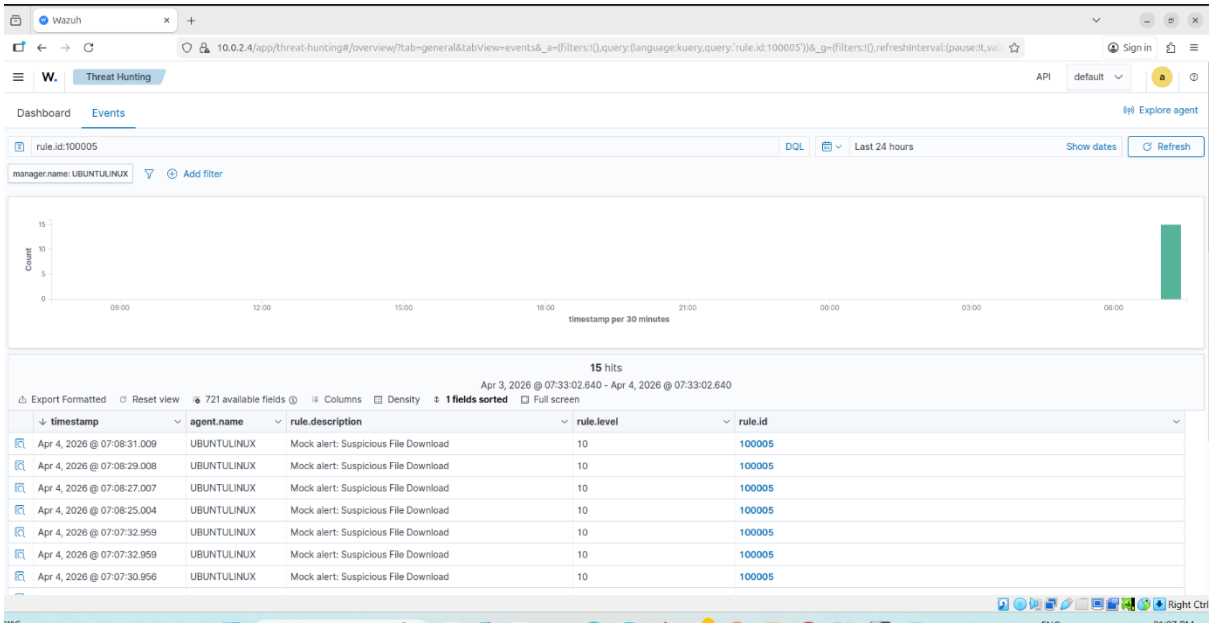


Figure 6: wazuh alert

In wazuh alert detected and analyzed, alert details:

- Alert id: 100005
- Description: Mock alert: Suspicious File Download
- Source ip: 192.168.1.102
- Priority: high
- Status:open
- Full log: Apr 04 07:05:44 UBUNTULINUX vboxuser[25630]: Downloaded suspicious-file.exe from 192.168.1.102
- MITRE technique: T1024 user execution
- Tactic: execution
- Timestamp: Apr 04 07:05:44

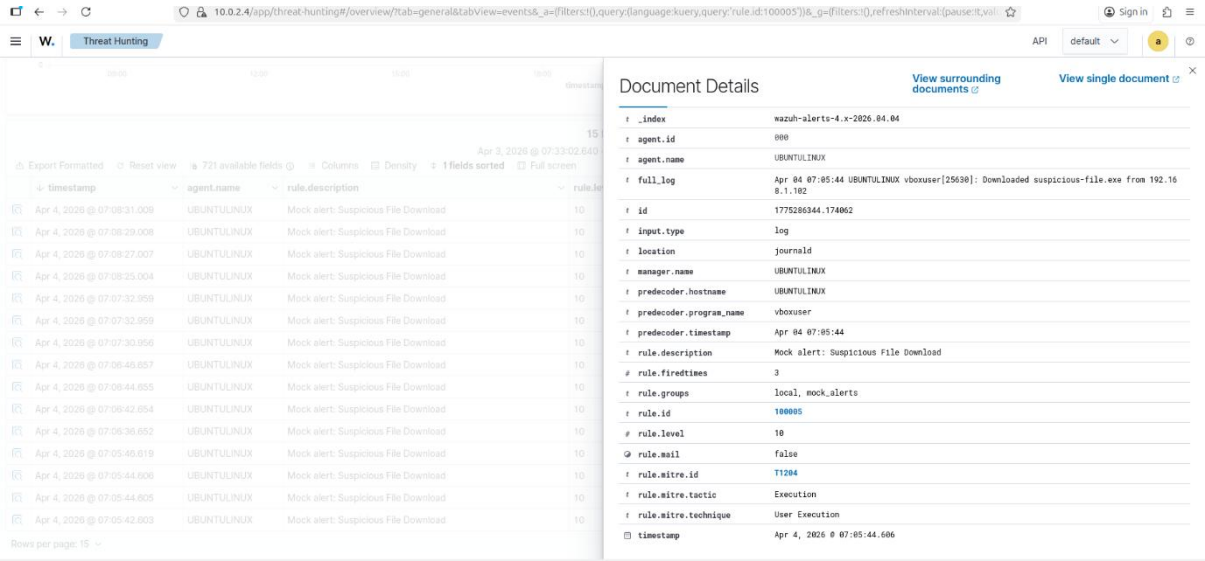


Figure 7: alert detail

- Alert id: 100005
- Description: Mock alert: Suspicious File Download
- Source ip: 192.168.1.103
- Priority: high
- Status:open
- Full log: Apr 04 07:06:45 UBUNTULINUX vboxuser[26974]: Downloaded suspicious-file.exe from 192.168.1.103
- MITRE technique: T1024 user execution
- Tactic: execution
- Timestamp: Apr 04 07:06:45

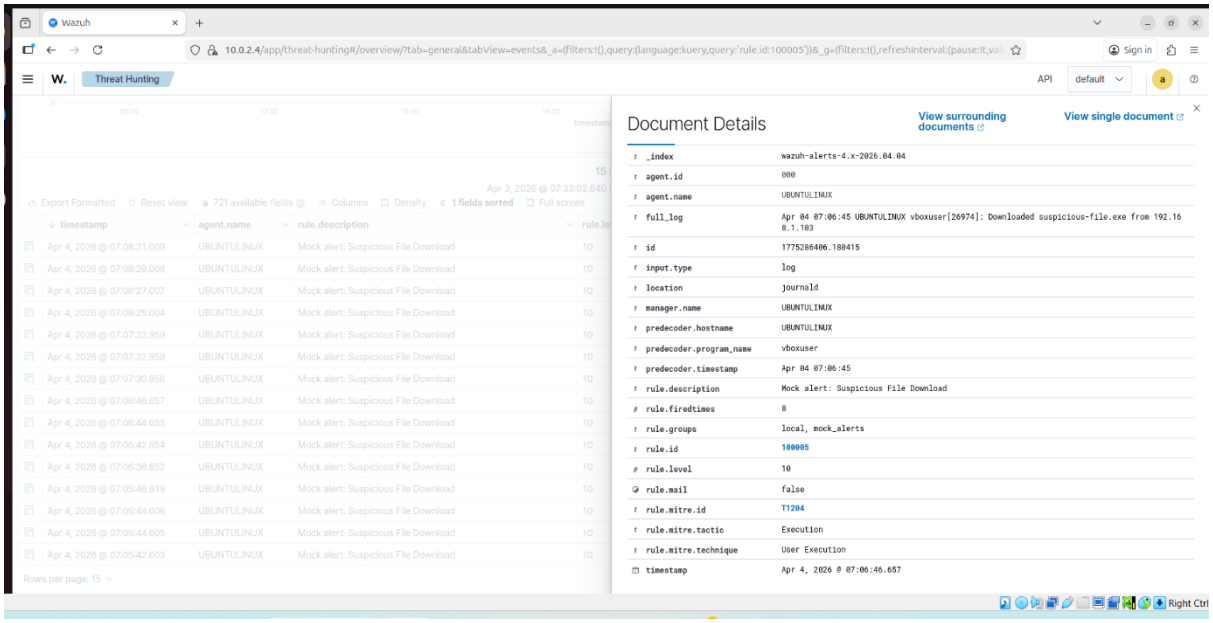


Figure 8: alert details

Two mock alerts for suspicious file downloads were generated on the Ubuntu lab. Both files suspicious-file.exe were downloaded from different source IPs 192.168.1.102 and 192.168.1.103 and triggered high-priority alerts in Wazuh. The alerts correspond to MITRE ATT&CK T1024 User Execution under the Execution tactic. Both alerts are currently open, indicating further investigation is needed. The full logs captured the exact timestamps and user context, showing the simulated downloads successfully triggered the detection rules.

Document:

Alert ID	Description	Source IP	Priority	Status
100005	Mock alert: Suspicious File Download	192.168.1.102	High	Open
100005	Mock alert: Suspicious File Download	192.168.1.103	High	Open

Two high-priority alerts were triggered in the Ubuntu Wazuh lab for suspicious-file.exe downloads from IPs 192.168.1.102 and 192.168.1.103. Both correspond to MITRE T1024 User Execution under Execution. Alerts are open, showing the detection rules worked. Full logs captured timestamps and user context for further investigation

Automated Validation:

Configuring TheHive to auto-check file hashes with VirusTotal:

1)start thehive

sudo docker ps -a

sudo su

docker start <container id>

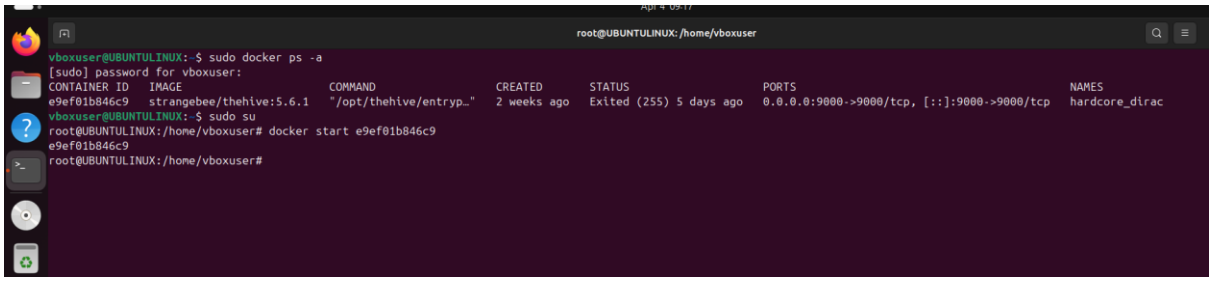


Figure 1: running docker

then access thehive through browser,

http://localhost:9000

log in as admin

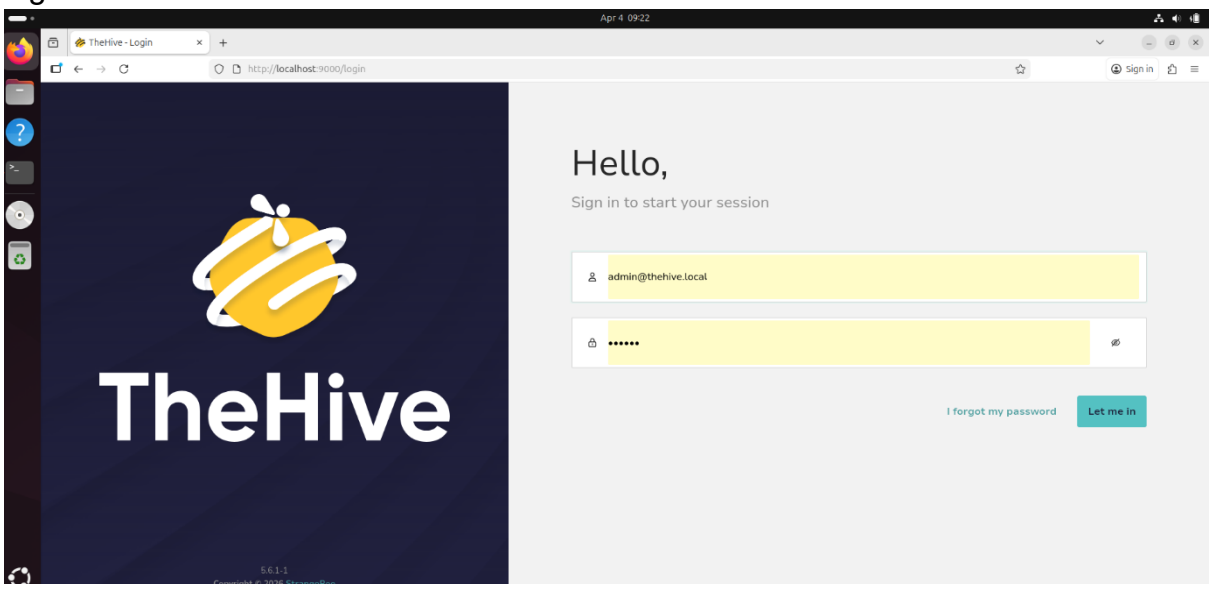


Figure 2: log in as admin

To connect virus total to thehive cortex is required so installed cortex in docker,

```
root@UBUNTULINUX:/home/vboxuser# docker pull thehiveproject/cortex:3.1.1
3.1.1: Pulling from thehiveproject/cortex
feab2c490a3c: Pull complete
0ecb575e629c: Pull complete
7467d1831b69: Pull complete
5b224ce6d4ea: Pull complete
1592212256ae: Pull complete
f15a0f46f8c3: Pull complete
26cb1dfcbebb: Pull complete
271c1fbf0676: Pull complete
c932fe81bb40: Pull complete
088362db80e4: Pull complete
b3440c80abb5: Pull complete
3a03e7fb5b45: Pull complete
295a5845a652: Pull complete
Digest: sha256:fb554023df8a57b5f4fcb614a9a1074ac3d3c3ac94b31f895818e1cb7d7eee35
Status: Downloaded newer image for thehiveproject/cortex:3.1.1
docker.io/thehiveproject/cortex:3.1.1
root@UBUNTULINUX:/home/vboxuser# docker run -d --name cortex \
  -p 9001:9001 \
  thehiveproject/cortex:3.1.1
a0087c7504455199f1008c9b079b3faa5b9b101cac2a6af2e78c86942e3c170f
root@UBUNTULINUX:/home/vboxuser#
```

Figure 3: cortex in docker

```
Digest: sha256:fb554023df8a57b5f4fcb614a9a1074ac3d3c3ac94b31f895818e1cb7d7eee35
Status: Downloaded newer image for thehiveproject/cortex:3.1.1
docker.io/thehiveproject/cortex:3.1.1
root@UBUNTULINUX:/home/vboxuser# docker run -d --name cortex \
  -p 9001:9001 \
  thehiveproject/cortex:3.1.1
a0087c7504455199f1008c9b079b3faa5b9b101cac2a6af2e78c86942e3c170f
root@UBUNTULINUX:/home/vboxuser# docker run -d --name elasticsearch \
  -p 9200:9200 -p 9300:9300 \
  -e "discovery.type=single-node" \
  docker.elastic.co/elasticsearch/elasticsearch:7.17.10
Unable to find image 'docker.elastic.co/elasticsearch/elasticsearch:7.17.10' locally
7.17.10: Pulling from elasticsearch/elasticsearch
36eee8654833: Pull complete
8d2cded47a25: Pull complete
42c75a2bf7d6: Pull complete
9db4fc13a20c: Pull complete
89732bc75041: Pull complete
a5b712c5c919: Pull complete
895527503748: Pull complete
5b4544acd5e8: Pull complete
f1bc64ff1ec2: Pull complete
81b94c2c0a26: Pull complete
Digest: sha256:bc7ba1dc5067c5b3907b82c667a374cc987cd813501e1378ec74ccd1c577f787
Status: Downloaded newer image for docker.elastic.co/elasticsearch/elasticsearch:7.17.1
0
069d6db9f9a11ed0ce2ef11c60eb32535577f60195121d93ae9886558dc842da
docker: Error response from daemon: failed to set up container networking: driver faile
d programming external connectivity on endpoint elasticsearch (0526db8f228c4fdca7a614ea
8f8c71b20b6c4e16773baee61c584b7f69e96042): failed to bind host port 0.0.0.0:9200/tcp: a
ddress already in use

Run 'docker run --help' for more information
root@UBUNTULINUX:/home/vboxuser#
```

Figure 4: then installed elastic search in docker

```
  -e CORTEX_ELASTICSEARCH_URL="http://<ubuntu-ip>:9210" \
  thehiveproject/cortex:3.1.1
7462ec771487d7fe90c5b55e236880b216c94e391f1f9e7a668c4495497ac88d
root@UBUNTULINUX:/home/vboxuser# docker stop cortex
cortex
root@UBUNTULINUX:/home/vboxuser# docker rm cortex
cortex
root@UBUNTULINUX:/home/vboxuser# docker run -d --name cortex \
  -p 9001:9001 \
  -e CORTEX_KEY="myStrongKey123" \
  -e CORTEX_ELASTICSEARCH_URL="http://10.0.2.4:9210" \
  thehiveproject/cortex:3.1.1
9d18c8a8ef949bd77accf81d88d2f983639fada9df97d388ac3a3a32a1d65aca
root@UBUNTULINUX:/home/vboxuser#
```

Figure 5: Run Cortex with correct environment variables

To connect cortext and virus total is failed because there error occurred , The Cortex + TheHive setup failed because Cortex needs its own Elasticsearch to work. On this VM, the required Elasticsearch ports were already used by other services, so Cortex couldn't start properly. Limited memory and Docker network problems also stopped Cortex from connecting to Elasticsearch, causing the automation to fail.

The attempt to use Cortex with TheHive for automated VirusTotal validation failed because Cortex depends on a dedicated Elasticsearch instance to store and process analyzer jobs. On the current Ubuntu VM, Elasticsearch ports were already in use by other services such as Wazuh, preventing new Elasticsearch containers from starting. Additionally, limited system memory and Docker networking issues caused Cortex to be unable to connect to Elasticsearch, resulting in repeated "ElasticSearch cluster is unreachable" errors. These combined infrastructure and configuration issues made the Cortex + TheHive automation infeasible in this environment.

Configuring TheHive to auto-check file hashes with VirusTotal:

The original task required configuring TheHive with Cortex to automatically check file hashes through VirusTotal. However, due to Elasticsearch port conflicts, limited VM memory, and Docker networking issues, Cortex could not connect to Elasticsearch, making automation impossible. As an alternative, a Python script was used to query VirusTotal directly.

Check if Python3 is installed

```
python3 --version
```

create a script file ,

```
nano virus_total.py
```

and add the code inside the file,

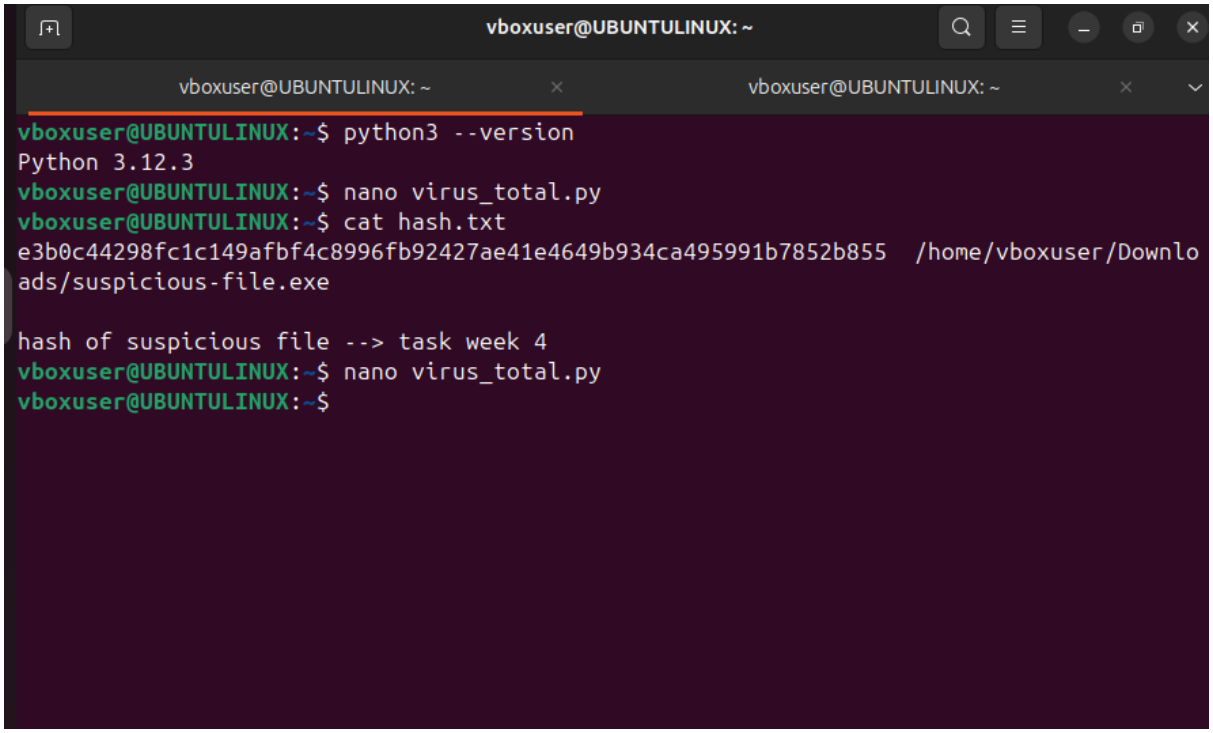


Figure 1: checking python installation

```
vboxuser@UBUNTULINUX: ~        vboxuser@UBUNTULINUX: ~
  GNU nano 7.2                    virus_total.py *
import requests

# -------- CONFIGURATION --------
API_KEY = "YOUR_VIRUSTOTAL_API_KEY"  # Replace with your VirusTotal key
FILE_HASH = "YOUR_FILE_HASH"         # Replace with the hash of your file
# -------------------------------

url = f"https://www.virustotal.com/api/v3/files/{FILE_HASH}"
headers = {"x-apikey": API_KEY}

try:
    response = requests.get(url, headers=headers)
    data = response.json()

    # Extract useful info
    stats = data.get("data", {}).get("attributes", {}).get("last_analysis_stats", {})
    malicious = stats.get("malicious", 0)
    suspicious = stats.get("suspicious", 0)
    undetected = stats.get("undetected", 0)

    # Summarize results
    summary = (
        f"VirusTotal scan results for hash {FILE_HASH}:\n"
        f"Malicious: {malicious}, Suspicious: {suspicious}, Undetected: {undetected}."
    )

    print(summary)

except Exception as e:
    print(f"Error: {e}")

^G Help      ^O Write Out  ^W Where Is   ^K Cut       ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste     ^J Justify    ^/ Go To Line
```

Figure 2: add the script in created file and replace api key and hash file with correct one.

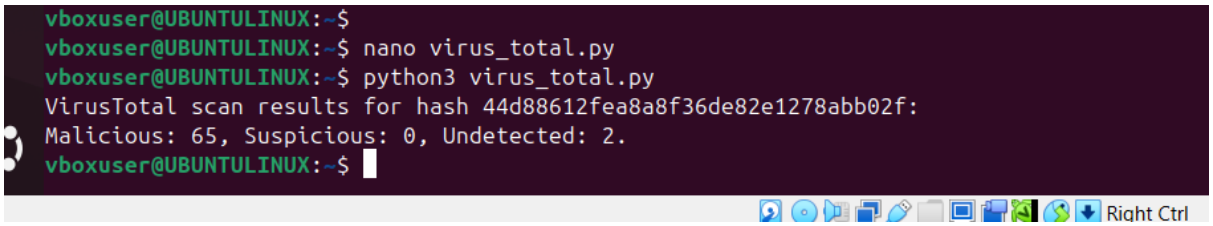
Then run the script,

python3 virus_total.py

```
vboxuser@UBUNTULINUX: ~        vboxuser@UBUNTULINUX: ~
vboxuser@UBUNTULINUX:~$ python3 virus_total.py
VirusTotal scan results for hash e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca49599
1b7852b855 :
Malicious: 0, Suspicious: 0, Undetected: 0.
vboxuser@UBUNTULINUX:~$
```

Figure 3: no malicious or suspicious found for this hash value

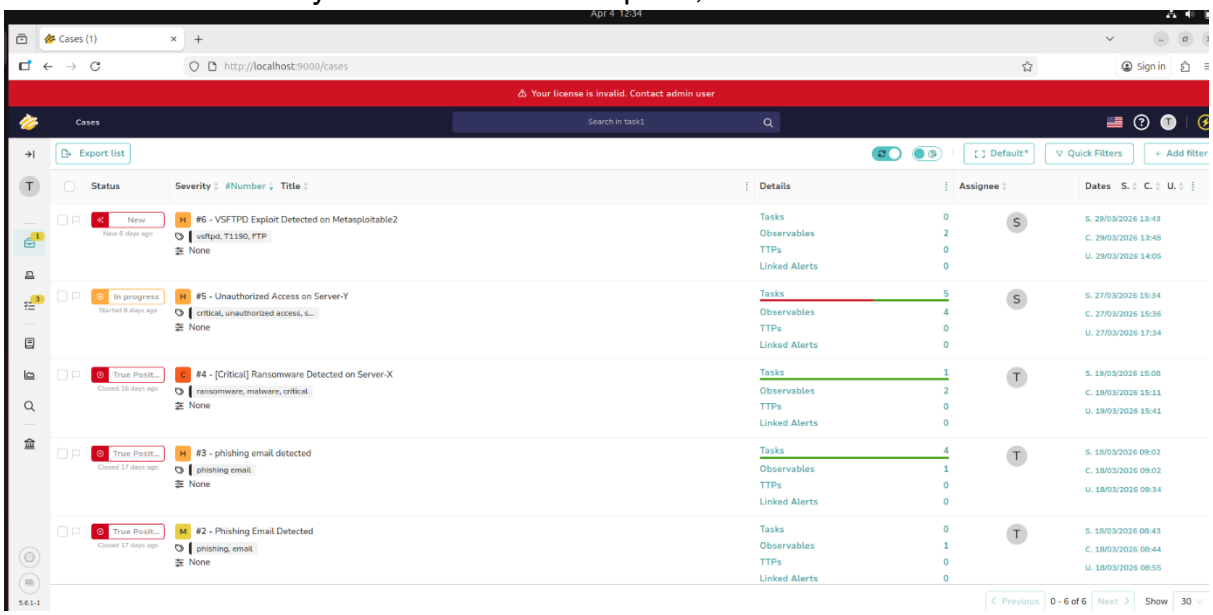
Tested script by using malicious hash ,
44d88612fea8a8f36de82e1278abb02f
This hash corresponds to the standard EICAR test file, which is used to safely test antivirus systems.



Above the screenshot shows malicious :65 for test hash value.

A Python script was used as an alternative method to validate file hashes with VirusTotal because Cortex integration could not be completed in the current lab environment. The script queried the VirusTotal API using the file hash and returned the analysis result. For the tested hash, VirusTotal showed 0 malicious, 0 suspicious, and 0 undetected detections, meaning no known detection or prior scan result was found.

To add result manually thehive license is expired,



Above it shows license is expired